Database Design Report

ProGineer Technical Assessment | Part 5

1. Introduction

The project uses MySQL to design a multi-vendor e-commerce database. The thirteen tables manage customers, addresses, profiles, vendors, products, categories, orders, order items, payments, shipments, reviews, and price history. Each entity has its own table, and foreign keys connect the related records to maintain data integrity.

2. Relationships

One-to-one (1:1)

Customers to CustomerProfiles: CustomerProfiles.id is both a primary key and a foreign key to Customers.id. This allows each customer to have at most one profile. The profile is optional, so a customer can exist without a matching profile record.

One-to-many (1:N)

- Customers to Orders: a customer can place multiple orders.
- Customers to CustomerAddresses: a customer can save several addresses.
- Vendors to Products: one vendor can offer many products.
- Orders to OrderItems: an order can contain several product items.
- Products to Reviews: a product can receive multiple customer reviews.
- Products to PriceHistory: a product can have many recorded price changes.
- Orders to Payments and Shipments: one order can have more than one payment or shipment record.

Many-to-many (M:N)

Products to Categories: ProductCategories is a junction table with the composite primary key (idCategories, idProducts). Its foreign keys connect both tables, allowing products to belong to several categories while preventing duplicate product-category pairs.

3. Normalization and historical data

First Normal Form (1NF): Each column stores one value. Product categories are linked through ProductCategories instead of storing several category IDs in one field.

Second Normal Form (2NF): Most tables have a single-column primary key. ProductCategories uses a composite key with no extra attributes that depend on only part of the key.

Third Normal Form (3NF): Customer, vendor, and product details are kept in separate tables rather than copied into orders. This reduces repeated information and helps avoid update problems.

Historical prices: OrderItems.unitPrice stores the selling price at purchase time and remains unchanged when Products.price changes. PriceHistory stores dated product prices. Products.price is kept for the current price, which is intentional duplication and must be updated consistently with the price history.

4. Constraints and data types

Feature	Reason / implementation
Primary keys	Integer IDs identify most records. ProductCategories uses a composite key, while CustomerProfiles shares its ID with Customers.
Foreign keys	Connect orders, customers, products, vendors, payments, shipments, reviews, and price history to valid parent records.
UNIQUE	Customer and vendor emails are unique. Category names and each (idOrder, productId) pair in OrderItems are also unique.
CHECK	Prevent negative product prices and stock, non-positive order quantities or payments, ratings outside 1-5, invalid statuses, and inconsistent shipment dates.
DEFAULT / NOT NULL	Require essential values, initialize product stock to zero, use Pending as the default status, and set some event dates automatically.
DECIMAL(10,2)	Stores product prices, purchase prices, historical prices, and payment amounts without floating-point rounding issues.
INTEGER / VARCHAR / TEXT	Integer IDs and quantities, VARCHAR for short text such as names and emails, and TEXT for comments and descriptions.
DATE / DATETIME	DATE is used for date of birth, while DATETIME records orders, reviews, shipments, and product price changes.

5. Indexing and scalability

Customer order lookups

idx_orders_customer_date on Orders(idCustomer, orderDate) groups orders by customer and date. This is useful when the number of customer orders grows.

Vendor product listings

idx_products_vendor_name on Products(idVendor, name) supports finding products belonging to a specific vendor and accessing them by product name.

Product reviews

idx_reviews_product_rating on Reviews(idProducts, rating) supports accessing the ratings recorded for a particular product.

Historical product prices

idx_pricehistory_product_date on PriceHistory(idProducts, changedAt) groups price records by product and change date, making each product's price history easier to access.

Careful index selection

Indexes improve access to frequently used fields but also consume storage and add work to inserts and updates. For small tables, MySQL may still choose to scan the table.

6. Design limitations and future improvements

Orders currently does not store a fixed shipping address, and Shipments is linked to an order but not to the specific products or vendors being shipped. A more detailed shipping design would be helpful when products from one order are delivered separately.

Products.price and PriceHistory.price must stay consistent. Updating both within one transaction would reduce the risk of saving a new current price without a matching history record. The current Payments relationship also allows multiple records per order, which can support separate payments or payment attempts.

7. Conclusion

The database includes the required relationship types, keeps the main business data in separate tables, and applies keys and constraints to protect data integrity. Its data types and selected indexes provide a practical base for the multi-vendor e-commerce system.